



AutoSortBin: Integrating CPS and IoT with Densely Connected Convolutional Networks for Sustainable Waste Management

Mayank Sangwan¹ · Avin Chaudhary¹ · Ayush Gupta¹ · Himank Gupta¹ · Vinay Kharb¹ · Shweta Sharma¹ · Sweeti Sah¹

Received: 1 May 2025 / Revised: 13 October 2025 / Accepted: 25 October 2025
© The Author(s) 2025

Abstract

Our proposed architecture, AutoSortBin, is based on the Cyber Physical System (CPS) and Internet of Things (IoT) technology which emphasizes the importance of proper waste disposal and segregation. To tackle the problem of waste segregation and management, AutoSortBin automatically classifies and sorts the waste into six broad categories namely metal, paper, plastic, glass, organic, and e-waste. The technical idea behind this segregation is implemented by the DenseNet-121 model for transfer learning integrated into the Wokwi simulator using ThingSpeak. The proposed framework takes an image input through a camera and identifies the waste as one of the main categories. The output from the waste identification model serves as an input for the IoT-based circuit and it opens the corresponding waste bin lid via servo motors. It also uses ultrasonic distance sensor to monitor the storage level in the waste bins. The proposed framework demonstrates an automation system that alerts the authorities to empty a bin whenever a bin is full by sending emails, to solve the problem of waste management efficiently thus contributing to environmental sustainability. The proposed framework has high potential for scalability and integration with different CPS and IoT platforms for enhanced performance and features. The setup exhibited an exceptional accuracy of 94.63% across the dataset collected.

Keywords IoT · Cyber physical system · DenseNet-121 · Transfer learning · Wokwi · ThingSpeak · Ultrasonic distance sensor · Sustainable waste management

Introduction

This section shows motivation, problem statement, contributions, and technical overview of our proposed framework:

Motivation

Waste generation is inevitable as it is a by-product of most human activities. Whether from industrial, agricultural, or domestic sources, the rate of waste production is increasing exponentially. According to the World Bank, over 2 billion tons of waste are generated annually, and this is expected to rise to 3.88 billion tons by 2050 [1], driven largely by rapid urbanization and industrialization, especially in developing countries. Improper waste treatment leads to pollution, accelerates climate change, and contributes to health problems. Despite efforts, global recycling rates remain poor; for example, only 9% of plastic waste is recycled, while 79% ends up in landfills or the environment [2]. This highlights the urgent global need for improved waste management practices to reduce environmental harm and improve sustainability.

✉ Mayank Sangwan
mayanksangwan0803@gmail.com

Avin Chaudhary
avinchaudhary1007@gmail.com

Ayush Gupta
ayushgupta0915@gmail.com

Himank Gupta
forhimankgupta@gmail.com

Vinay Kharb
vinaykharb18@gmail.com

Shweta Sharma
shweta.sharma@nitkkr.ac.in

Sweeti Sah
sweetisah3@nitkkr.ac.in

¹ Department of Computer Engineering, National Institute of Technology Kurukshetra, Haryana, India

Problem Statement

A major obstacle in improving waste management is the lack of efficient and reliable waste segregation. Manual segregation is time-consuming, inconsistent, and often ignored, leading to lower recycling rates and more waste ending up in landfills. There is a pressing need for an automated system capable of accurately sorting waste in real time without human involvement. Such a system can play a crucial role in enhancing recycling efficiency, supporting environmental sustainability, and reducing the burden on existing waste management infrastructure.

Contributions

We have made following contributions to our proposed Cyber Physical System (CPS) and Internet of Things (IoT) enabled framework:

1. **CPS-Driven Waste Classification using Transfer Learning:** The proposed framework integrates DenseNet-121 with a CPS setup, combining AI-based image classification and physical actuation for waste segregation. It classifies waste into six types; Paper, Glass, Metal, Plastic, Organic, and Electronic Waste (E-Waste); within an IoT-enabled simulation environment using Wokwi and ThingSpeak. This fine-grained segregation improves recycling efficiency and promotes environmental sustainability.
2. **Dedicated E-Waste Handling:** Unlike basic systems, this framework uniquely identifies and separates E-Waste to make sure it's handled and processed safely. Proper disposal of hazardous electronic waste products is ensured reducing environmental risks.
3. **Robust Customized Dataset:** The dataset for the model contains 976 labeled images from various sources including one manually collected dataset. This elaborated and unique dataset helps the AI model become more accurate and reliable under diverse real-world conditions providing robust waste identification.
4. **Real-Time Email Alerts:** Whenever any bin is full, the system generates an email alert that is sent to the concerned authorities. This ensures that waste is collected on time to prevent overflow and proper hygiene is maintained at waste disposal locations.
5. **Smart Camera Detection:** The proposed framework captures real-time camera images to classify the waste category. This automation leverages AI and CPS synergy, significantly reducing manual labor and improving detection precision.
6. **CPS-IoT Integration for Bin Full Detection:** An ultrasonic distance sensor monitors the fill level of each bin.

Once full, the system disables further lid operation until the bin is emptied, preventing overfilling and potential hardware issues. This tightly coupled interaction of sensing, actuation, and communication embodies a complete CPS-IoT loop for efficient waste management.

Technical Overview

Our proposed framework, AutoSortBin, is based on an emerging CPS and IoT technology. It utilizes a pre-trained transfer learning model called DenseNet-121 [3], which is fine-tuned for waste classification. The model uses transfer learning, allowing it to efficiently process and identify images without requiring training from scratch. The circuit is simulated on an online IoT simulator Wokwi [4] by CodeMagic Ltd. ThingSpeak [5] cloud server is used to integrate the transfer learning model with the circuit simulation. Various components used in the experimental setup are ESP32 microcontroller, input devices such as camera and ultrasonic distance sensor (HC-SR04), output devices such as 16×2 LCD (I2C interface) and servo motors, and jumper wires and breadboard for connections.

Related Works

We give a qualitative comparison of the proposed framework with the related works in terms of presence of alert system and real-time monitoring system. A number of research efforts have focused on automating waste classification using different methods as shown in Table 1.

For instance, Fernandez et al. [6] built a smart bin using Arduino Uno along with ultrasonic, IR, and moisture sensors to separate wet and dry waste. Bonala et al. [7] used a MobileNet-based CNN model and achieved 90.21% accuracy in classifying waste as recyclable or non-recyclable. Sairam et al. [8] developed a Convolutional Neural Network (CNN)-powered system to sort biodegradable, non-biodegradable, and biomedical waste, reaching 91.87% accuracy.

Similarly, Song et al. [9] implemented EfficientNet-B0 to classify a wide range of waste types like food, paper, textile, plastic, and plant waste with 93.33% accuracy. Varudandi et al. [10] used machine learning to distinguish plastic, paper, medical, hazardous, metallic, and biodegradable waste, with 90% accuracy. Shamin et al. [11] achieved 99% accuracy by combining K-Nearest Neighbor (KNN) and Speeded-Up Robust Features (SURF) with sensor-based detection.

Das et al. [12] proposed a Raspberry Pi-based system for wet and dry waste sorting, while Mallikarjuna et al. [13] applied a CNN to classify organic and recyclable waste. Pereira et al. [14] used capacitive sensors to categorize

Table 1 Related contributions in the field of solid waste management

Author	Year	Waste Categories	Model/Technique	Sensors/Components Used	Alert System	Real-Time Monitoring	Accuracy	Gaps
Auto-SortBin (Proposed)	2025	Plastic, Paper, Glass, Metal, Organic, E-Waste	DenseNet-121 + CPS + IoT	ESP-32, Camera, Ultrasonic distance sensor (HC-SR04), 16x2 LCD (I2C interface), Servo Motors	✓(Email alerts)	✓	94.63%	Full automation, detailed classification, E-waste included
Fernandez et al. [6]	2024	Wet, Dry	Sensor-based sorting	Arduino Uno, Ultrasonic Sensor, IR sensor, Moisture sensor, Servo Motors, Wireless Modules	✗	✗	NA	Simple hardware model, lacks intelligence and adaptability
Bonala et al. [7]	2024	Recyclable, Non-recyclable	MobileNet	Arduino Uno R3, Servo Motor, Camera Module	✗	✗	90.21%	Basic classification, no real-time processing
Sairam et al. [8]	2024	Biodegradable, non-biodegradable, biomedical	CNN	Camera, Bins	✗	✗	91.87%	Broader classification, lacks feedback loop
Song et al. [9]	2023	Kitchen, meal, paper, textile, plastic, dust, plant, other	EfficientNet-B0	Time of flight sensor, DHT22, Load cell, Gas sensor	✗	Partial	93.33%	High category coverage, but ambiguity in visual classes
Varudandi et al. [10]	2021	Plastic, paper, medical, hazardous, metallic, biodegradable	Clarifai custom model	Camera Sensor (OV7670), Ultrasonic Sensor, Humidity Sensor (DHT-22), GPS, Arduino Uno	✗	✗	90.00%	Uses many sensors; complex implementation
Shamin et al. [11]	2021	wet, dry, metal, non-metal, degradable, non-degradable	Image Processing, KNN, SURF	Arduino mega, Moisture Sensor, Metal Sensor, Ultrasonic Sensor, servo motor	✗	✗	99.00%	Highest accuracy but high hardware complexity
Das et al. [12]	2021	Wet, dry	Sensor-only	Raspberry Pi 3, DHT11 Sensor (Humidity-based), Load Cell(CZL635), Ultrasonic Sensor(HC-SR04)	✗	✗	NA	Sensor-heavy, lacks AI capabilities
Mallikarjuna et al. [13]	2021	Organic, recyclable	CNN	Arduino, Servo Motor, Camera Module	✗	✗	90.00%	Basic deep learning model with limited classes
Pereira et al. [14]	2019	Plastic, wet, dry	Sensor-based	Capacitive plates, Bluetooth module, Servo motor, Ultrasonic sensor, IR Sensor, Stepper Driver, and Stepper Motor	✗	✗	NA	No ML integration, low flexibility

plastic, wet, and dry waste. These studies highlight different approaches using a mix of sensors and AI models for waste classification.

Research Gaps

Many current systems still rely heavily on sensors without smart adaptability, as seen in the works of Fernandez et al. [6] and Das et al. [12], making it hard for them to adapt to new or unexpected waste types. Bonala et al. [7] and Sairam et al. [8] focused on basic classification, but didn't go into

deeper categorization, which is essential for proper recycling. On the other hand, models from Varudandi et al. [10] and Shamin et al. [11] use many sensors, which make the system complex without always giving better results. Similarly, Mallikarjuna et al. [13] and Pereira et al. [14] handled only a few types of waste, limiting their usefulness. Many of these systems [9] also can't process data in real time, and they don't include automatic alerts, which can delay waste collection and lead to overflowing bins.

Most prior works either rely on coarse classification (wet/dry, recyclable/non-recyclable) or lack important categories

such as E-Waste. They are also often semi-automated, still needing people to handle waste manually.

We use a DenseNet-121 transfer-learning backbone fine-tuned on our custom dataset that achieved 94.63% validation accuracy and outperformed ResNet-50 and VGG-16 in our controlled benchmarks. Another major gap is the lack of real-time bin monitoring and alert systems; something our framework addresses with email alerts to ensure timely collection. By combining AI-based classification, live monitoring, and automation, our framework brings a smarter and more efficient way to manage waste.

Dataset Description

Our proposed framework, AutoSortBin, is trained using a diverse dataset sourced from three different places: two Kaggle datasets, i.e. Dataset 1 [15], Dataset 2 [16] and one created manually, i.e. Dataset 3. Dataset1 [15] had 2,527 images, all used for training. Dataset2 [16] had 15,515 images, from which we selected 6,447 for training. Additionally, we used 976 images from our own dataset, i.e. Dataset 3. Figure 1 shows sample images from the datasets, covering six different categories.

Figure 2 illustrates the distribution of images across the three sets, 70% for training, 15% for testing, and 15% for validation. It also presents the number of images from each of the six categories within each split. The train-test-validation split was done separately on each waste category. The total dataset (9950) consisted of e-waste (1523), glass

(1770), metal (1669), organic (1462), paper (1826) and plastic (1700) splits. The dataset was created in such a way that it will keep the class imbalance to minimum after merging with the other two datasets.

Preprocessing Pipeline

Before training the model, all images were subjected to a consistent preprocessing pipeline to ensure compatibility with the DenseNet-121 architecture and to enhance model performance. First, all images were resized to a resolution of 224×224 pixels, aligning with the input size requirements of DenseNet-121. Pixel values were then normalized to a $[0, 1]$ scale to stabilize training and accelerate convergence. If any image was originally in grayscale, it was converted to RGB format, as DenseNet expects three-channel color input. To reduce unwanted background noise and improve feature clarity, Gaussian Blur and Median Filtering were applied as denoising techniques.

To further improve generalization and prevent overfitting, various data augmentation techniques were applied to the training set. These included random rotations within ± 16 degrees, horizontal and vertical flipping, and zooming in or out by approximately 10%. Finally, class labels representing waste categories (e.g., paper, plastic, glass) were one-hot encoded to be compatible with the model's softmax output layer and categorical cross-entropy loss function.

An overview of these preprocessing steps is summarized in Table 2, which details each stage and its purpose in preparing the dataset for training and evaluation.



Fig. 1 Sample Images of six different categories from the dataset

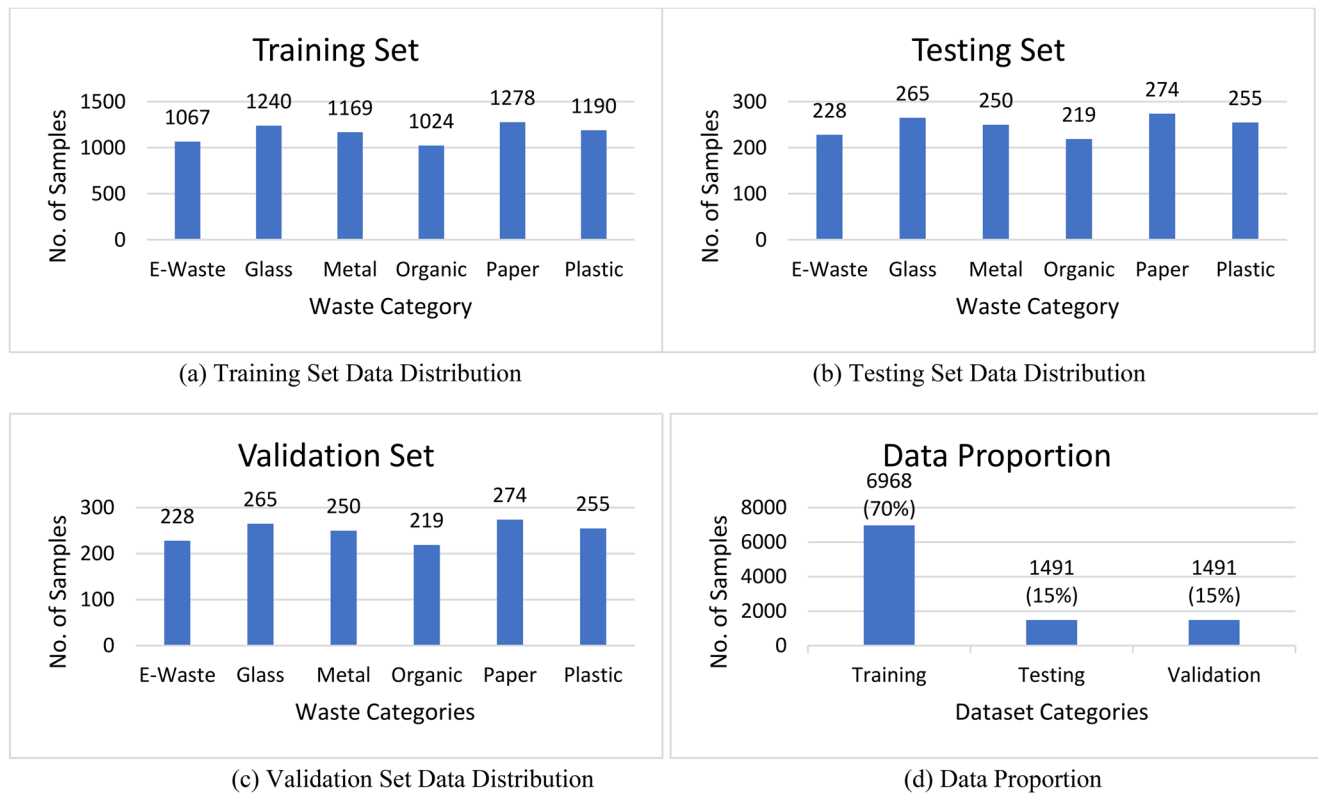


Fig. 2 Dataset distribution and proportion

Table 2 Image preprocessing steps

Step(s)	Description
Resizing	All images resized to 224×224 pixels to fit DenseNet-121 input requirements
Normalization	Pixel values scaled to $[0, 1]$ for stable training
Color Conversion	Grayscale images converted to RGB format
Noise Reduction	Gaussian Blur and Median Filter used to clean image noise
Data Augmentation	Rotation ($\pm 16^\circ$), flipping, and zoom applied to training images only
Label Encoding	Class labels converted to one-hot encoded vectors

Proposed AutoSortBin Framework

DenseNet-121 is a pre-trained architecture designed for image classification tasks [3]. Unlike traditional CNNs, where layers receive inputs only from their immediate predecessors, DenseNet-121 connects each layer to all previous layers within a block. This results in $L(L + 1)/2$ direct connections for L layers, ensuring that feature maps from earlier layers are reused throughout the network. In the proposed framework, a pre-trained DenseNet-121 model is fine-tuned for waste classification instead of training from scratch. All images from the ESP32 camera are securely sent to a cloud service running DenseNet-121 for classification. The result is then sent to ThingSpeak. Cloud inference allows using heavier models, simplifies updates, and handles processing beyond the ESP32's capacity. This transfer learning approach enhances feature propagation while significantly

reducing computational costs and training time. By freezing initial layers and fine-tuning only the fully connected layers, the model adapts to the specific waste classification task efficiently.

When waste exceeds the threshold of 10 cm, the ESP32 sends data to ThingSpeak, triggering an email alert. Email was chosen because it's easy for everyone to access, doesn't require a separate application login, and provides a dependable record of activity. Unlike commercial systems (e.g., Bigbelly and Ecube Labs) that rely on dashboards and push notifications, this email-based approach is better suited for early-stage research setups.

Figure 3 presents the architecture of our proposed AutoSortBin framework. The corresponding control flow is outlined below:

1. The image of waste is captured by the camera.

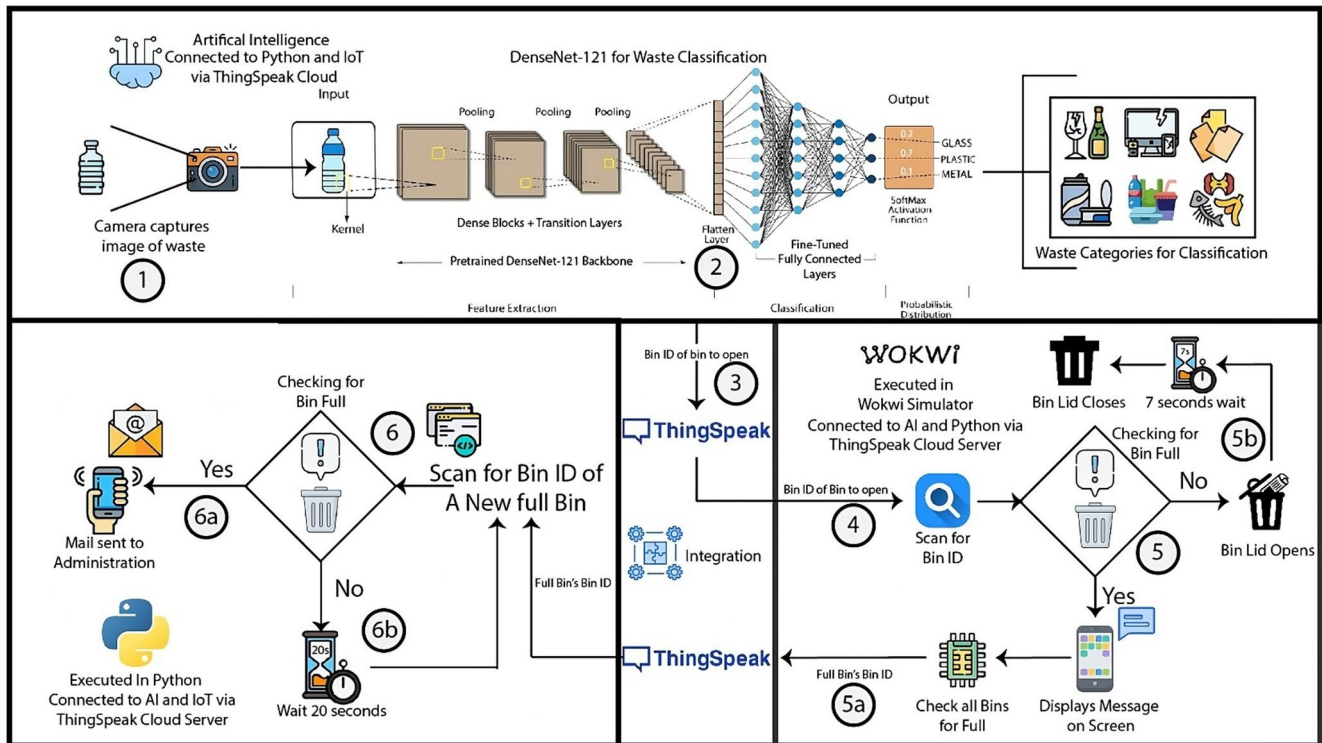


Fig. 3 Control Flow of the AutoSortBin Framework

2. The DenseNet model processes the image to classify the waste into one of six categories: glass, paper, plastic, metal, e-waste, or organic.
3. Bin ID of the bin associated with the waste category is generated and sent to the ThingSpeak server.
4. The server then interacts with the Wokwi platform.
5. Bin capacity is monitored by HC-SR04 ultrasonic sensors calculating fill percentage from echo pulse duration.
 - a. If any bin is full:
 - i. An LED display shows “Bin is Full.”
 - ii. The Bin ID of full bin is sent again to ThingSpeak server.
 - b. If any bin is not full:
 - i. The ESP32 generates PWM control signals to drive servo motors, opening the lid of the bin mapped to the classified waste category.
 - ii. The system waits for 7 s before closing the lid.
6. An independent module checks every 20 s whether ThingSpeak has any new bin id (for bin that is full).
 - a. If any new bin is full:
 - i. The ESP32 triggers a ThingSpeak REST API call to initiate an automated email alert.
 - ii. ThingSpeak server’s data value is updated to negate the new full bin id’s value.
 - b. If any new bin is not full:
 - i. Wait 20 s and check again.

Algorithm 1 uses a DenseNet-121 model to classify waste for AutoSortBin. It starts by cleaning and resizing images, then boosts the dataset with augmented versions to improve accuracy. The data is split into training, validation, and test sets. The model is trained using the Adam optimizer, with early stopping if it stops improving. Finally, the best model is tested on unseen data and evaluated using accuracy, precision, recall, and F1-score to check how well it classifies waste.

Algorithm 1: DenseNet-121 for Waste Management in AutoSortBin

Inputs: Waste images IMG, corresponding labels, hyperparameters, and DenseNet configuration

Output: Trained DenseNet-121 model and Predicted waste categories

Preprocessing Waste Images

```
for each image IMG[i] in dataset do
    Resize IMG[i] to (224 × 224)
    Normalize pixel values to range [0,1]
    Convert to RGB if grayscale
    Apply Gaussian Blur or Median Filter for noise reduction
```

end for

Data Augmentation for Improved Generalization

Initialize list AugmentedData = []

```
for each image IMG[i] in dataset do
    Apply random transformations: rotation, flipping, zooming, and shifting
    Append transformed image to AugmentedData
```

end for

Split Dataset into Train, Validation, and Test Sets

Split AugmentedData and Labels into: - Train (70%), Validation (15%), Test (15%)

Define DenseNet Model

Initialize DenseNet with:

- Input Layer of size (224 × 224 × 3)
- Multiple Dense Blocks with Batch Normalization and ReLU Activation
- Transition Layers with 1x1 Convolution and 2x2 Average Pooling
- Global Average Pooling Layer
- Fully Connected Layer with Softmax Activation

Compile model using:

- Optimizer: Adam with learning rate $\alpha = 0.00005$
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy

Train DenseNet Model

Initialize best model $\leftarrow$ NULL, best val loss $\leftarrow \infty$

```
for epoch = 1 to MaxEpochs do
    Perform forward pass on training data
    Calculate loss using backpropagation
    Update model weights using Adam optimizer
    Evaluate on validation set to calculate validation loss
    if validation loss < best val loss then
        Save model as best model
        best val loss  $\leftarrow$  validation loss
    end if
    if validation loss doesn't improve for patience threshold epochs then
        Perform Early Stopping
        Break
    end if
```

end for

Evaluate the Model

Load the best saved model (best model)

Perform prediction on Test set using the DenseNet-121 model

Calculate accuracy, precision, recall, and F1-score

Print evaluation metrics

Return Model Evaluation Metrics

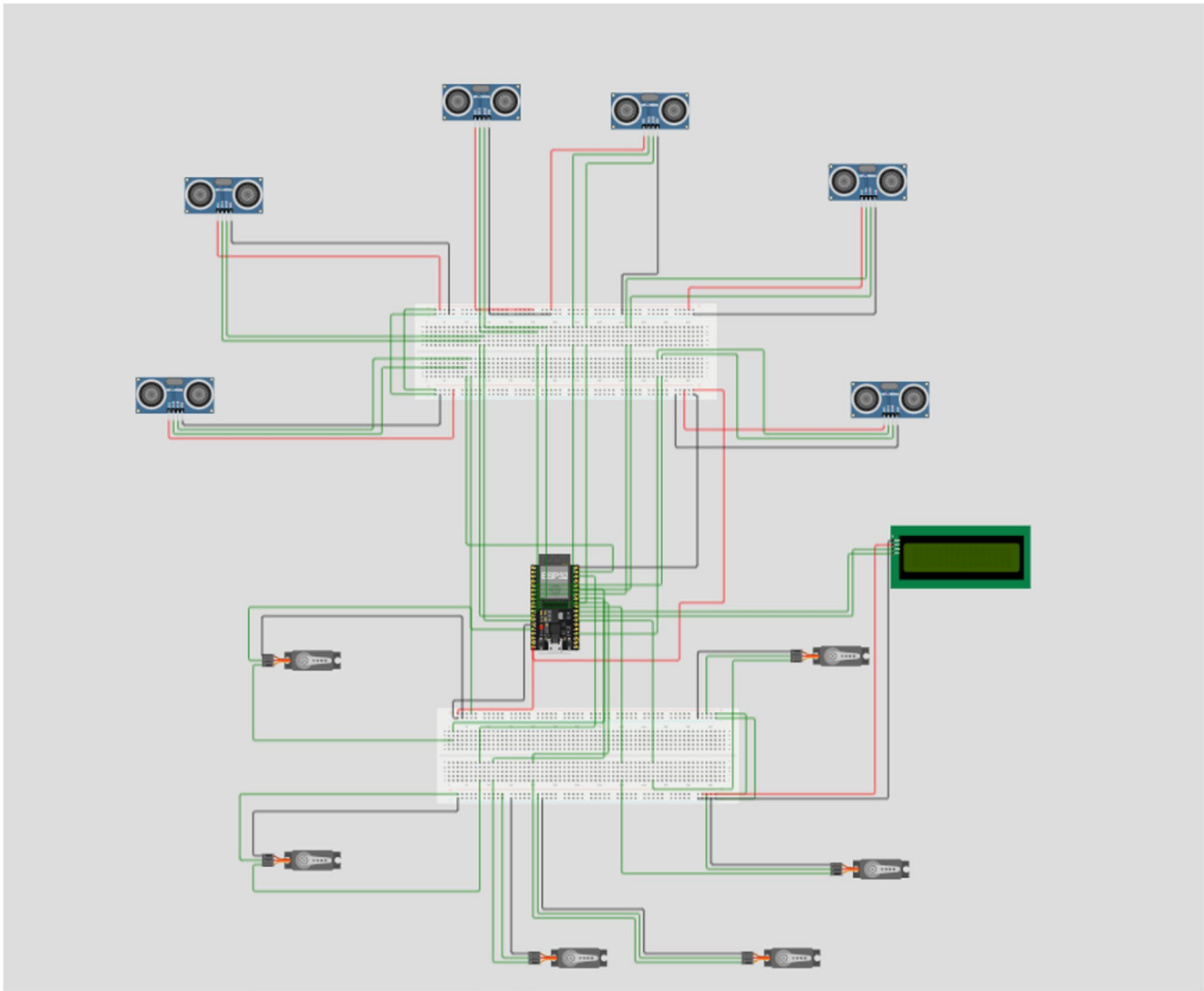


Fig. 4 Schematic representation of IoT Circuit on Wokwi Simulator

IoT Circuit for Proposed AutoSortBin Framework

Figure 4 represents the IoT circuit setup of the proposed CPS based AutoSortBin framework on the Wokwi simulator. The circuit consists of six HC-SR04 Ultrasonic Distance sensors that check the bin levels, six servo motors to control the lids of respective bins and one 16×2 LCD (I2C interface) to

display messages such as bin status. Everything is connected to the ESP32 microcontroller using breadboard and jumper wires.

The code snippets for functions of hardware implementation on the Wokwi simulator are shown as follows:


```

Servo servos[numBins];
const int servoPins[numBins] = {19, 22, 21, 18, 5, 4};
const int trigPins[numBins] = {23, 14, 27, 26, 32, 15};
const int echoPins[numBins] = {13, 12, 33, 25, 35, 34};
LiquidCrystal_I2C lcd(0x27, 16, 2);

bool isBinFull(int binIndex) {
    digitalWrite(trigPins[binIndex], LOW);
    delayMicroseconds(2);
    digitalWrite(trigPins[binIndex], HIGH);
    delayMicroseconds(10);
    long duration = pulseIn(echoPins[binIndex], HIGH);
    long distance = duration * 0.034 / 2;
    return distance <= binFullThreshold;
}

int getThingSpeakData() {
    HTTPClient http;
    String url = String(server) + "/channels/" + String(channelID) + "/fields/1.json?api_key=" + apiKey + "&results=2";
    http.begin(url);
    String payload = http.getString();
    DynamicJsonDocument doc(1024);
    deserializeJson(doc, payload);
    const char *field1_value2 = doc["feeds"][1]["field1"];
    return atoi(field1_value2);
}

void giveThingspeakBinno(int binNo) {
    HTTPClient http;
    http.begin(serverName);
    http.addHeader("Content-Type", "application/json");
    String httpRequestData = "{\"api_key\":\"" + writeapiKey + "\",\"field2\":\"" + String(binNo) + "\"}";
    http.POST(httpRequestData);
}

```

Real-Time Example

Figures 5 and 6 show the working examples where Fig. 5(a) and (b) represents the images of paper and metal waste being identified by the AutoSortBin framework in real-time. Figure 6 shows the output message on the LCD screen for warning about bin 2 being full, waste category detected, and respective and the respective bin being opened upon detection of the waste.

Integration with ThingSpeak

ThingSpeak acts as a bridge between our AI-based waste classification running in the cloud and the ESP32 microcontroller (simulated using Wokwi) that controls the bin's movements. The whole system works in three steps: whenever an item is classified, its category label is sent to field1 of our ThingSpeak channel. If the bin gets full, its ID is sent to field2, which also triggers an automatic email alert. Meanwhile, the ESP32 keeps checking these fields, opening

the right bin when a new category shows up in field1 or handling full-bin alerts when field2 updates.

We chose ThingSpeak because it makes IoT prototyping easier by offering multiple fields to organize data, built-in charts for live monitoring, and a simple API. Plus, its free academic plan fits our project's needs well. However, ThingSpeak needs a 15-second gap between updates, so we had to spread out data writes to avoid errors. Also, because the ESP32 checks for updates at regular intervals, there's a slight delay usually around 10 to 15 s before it reacts.

After classification, AutoSortBin sends the bin ID to ThingSpeak via secure HTTPS, and the ESP32 (Wokwi-simulated) polls the channel to receive commands. In our framework, the full loop of cloud write, ThingSpeak processing, and device polling gives an end-to-end reaction time of about 8–15 s in most cases, with occasional faster responses (~3–5 s) when the polling cycle aligns, which is reasonable for supervised indoor or public bins. To maintain reliability, we verify HTTP responses, retry failed requests with exponential backoff, and

buffer unsent events locally on the ESP32. Security is ensured through HTTPS, channel-level API keys (kept private and rotated), and validation of incoming values. For scalability, individual or grouped channels can be used for multiple bins, and in city-wide setups, lightweight edge devices or brokers can collect and forward data to the cloud. This keeps communication efficient and avoids hitting channel limits.

Algorithm 2: Model Integration Workflow with ThingSpeak

Inputs: Captured waste image, fine-tuned DenseNet-121 model, ThingSpeak API Key
Output: Automated Bin ID triggering and hardware feedback via ThingSpeak and Wokwi

```

# Preprocessing Image for Classification
Capture image via webcam or image upload interface
Resize image to (224 × 224)
Normalize pixel values to range [0, 1]
Apply Gaussian Blur to reduce noise
# Model Inference for Waste Classification
Load pre-trained and fine-tuned DenseNet-121 model
Perform forward pass on preprocessed image
Extract predicted class via arg max on output logits
Map class label (0–5) to corresponding BinID
# IoT Integration using HTTP API
Construct HTTP POST request to ThingSpeak channel
Include BinID in designated field using API Key
Send request and await server acknowledgment
Log status response and timestamp for tracking
# Continuous Bin Monitoring and Actuation
while True do
    Fetch latest BinID from ThingSpeak every 5 seconds
    if new BinID is detected then
        Read ultrasonic sensor for bin fullness
        if Bin is not full then
            Trigger corresponding servo motor to open bin lid
            Display "Bin #BinID Opened" on 16x2 LCD
            Wait for 7 seconds
            Close the bin lid via servo motor
        end if
    else
        Update ThingSpeak field with bin-full status
        Trigger email alert via ThingSpeak Webhooks
    end else
end if
Wait 5 seconds before next read cycle
end while
Return Bin Activation Feedback and System Logs

```

Algorithm 2 connects the waste classification model with IoT hardware using ThingSpeak. It starts by capturing and preprocessing an image, which is then classified using the DenseNet-121 model to determine the waste type and map it to a Bin ID. This ID is sent to ThingSpeak via an HTTP request using an API key. ESP32 periodically queries the ThingSpeak channel via HTTP GET requests to retrieve the latest classification result. When a bin is triggered, it checks if it's full using an ultrasonic distance sensor. If not, it opens the bin using a servo motor, displays the action on an LCD, and then closes the lid. If the bin is full, it logs the status and sends an alert using ThingSpeak webhooks.

Experimental Setup

The proposed AutoSortBin framework utilizes various hardware components such as sensors, motors and output devices as well as various software for simulation,

integration and transfer learning to accomplish its functioning. Servo motors are used as bin lids, which open precisely based on the output of waste classification model for quick and accurate waste disposal. Their low power use and connection with the ultrasonic sensor help prevent overflow, making the system efficient. Servo motors are powered by a dedicated 5 V rail, HC-SR04 echo lines are level-shifted to 3.3 V for ESP32 GPIO protection. The ultrasonic distance sensor plays a key role in checking each bin's fill level. Every bin has an HC-SR04 sensor that measures the distance from the sensor to the top of the waste. When this distance drops below the set threshold of 10 cm, the ESP32 identifies the bin as full. It then disables the corresponding lid to stop further waste from being added and sends a status update to ThingSpeak, which triggers an automated email alert to the waste management authorities with the bin ID and timestamp. The overview of all the hardware and software components used in the experimental setup is described in Tables 3 and 4 respectively.

Experimental Results

Table 5 contains the experimental results on training, testing, and validation data in terms of Precision, Recall, F1-score, and Accuracy defined in equation (i), (ii), (iii), and (iv), respectively [17]. This experiment was conducted on single item per input testing scenario. Precision indicates how many of the predicted positive instances are actually positive, helping to reduce false positives. Recall measures how many actual positive instances were correctly identified, reducing false negatives. F1-score is the harmonic mean of precision and recall, balancing both when there is an uneven class distribution. Accuracy measures the overall correctness of predictions as the ratio of correctly classified instances to total instances.

These metrics are based on four key values: True Positives (TP), where positive instances are correctly classified; True Negatives (TN), where negative instances are correctly classified; False Positives (FP), where negative instances are incorrectly classified as positive (Type I error); and False Negatives (FN), where positive instances are incorrectly classified as negative (Type II error).

$$Precision = \frac{TruePositive}{TruePositive + Falsepositive} \quad (1)$$

$$Recall = \frac{TruePositive}{TruePositive + FalseNegative} \quad (2)$$

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (3)$$



(a): Paper waste predicted by the model

(b): Metal waste predicted by the model

Fig. 5 (a): Paper waste predicted by the model (b): Metal waste predicted by the model**Fig. 6** Message displayed on LCD corresponding to Bin status

$$\text{Accuracy} = \frac{\text{TruePositive} + \text{TrueNegative}}{\text{TruePositive} + \text{TrueNegative} + \text{FalsePositive} + \text{FalseNegative}} \quad (4)$$

While Table 6 summarizes the model's overall performance, a more granular analysis is necessary to validate its effectiveness across each distinct waste category. To this end, we evaluated the model on the validation set to generate per-class performance metrics and a detailed confusion matrix. Figure 7 shows the confusion matrix of the DenseNet-121 Model on the validation set.

The model is trained using a batch size of 40 for 20 epochs with a learning rate of 0.00005, optimized using the Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$). The categorical cross-entropy loss function is used for multi-class classification. The architecture includes a dense layer with 128 units and ReLU activation function, followed by an output layer with 6 units and a softmax activation function. Data augmentation techniques such as horizontal flip, vertical flip, and rotation up to 16 degrees are applied to improve generalization.

Real-Time Waste Detection

After training the model with exactly 70% of the dataset, half of the remaining dataset was used for testing (15%) and subsequent half for validation (15%). Table 7 demonstrates the output of the sample images from the Dataset 3 given in real-time.

Comparison with Other Models and Existing Frameworks

While deeper architecture like ResNet-50 and VGG-16 can achieve higher accuracies, they do so with significantly higher parameter counts as compared to DenseNet-121, which are not efficient in CPS and IoT-based systems [18]. To empirically validate our choice of DenseNet-121, we conducted a direct comparison against ResNet-50 and VGG-16 under identical training conditions. All models were trained on Google Colab using an NVIDIA Tesla T4 GPU. As shown in Table 8, DenseNet-121 outperformed both alternatives, achieving an F1-score of 94.4% and the highest accuracy of 94.6% on the validation set. It also demonstrated approximately 12% faster

Table 3 Hardware details

Components	Model	Function
Micro-controller	ESP32	Responsible for processing data
LCD display	LCD 16 × 2	Displays the message of current status
Servo motor	-	Opens and closes the lid of the respective bin.
Ultrasonic sensor	HC-SR04	Detects whether the bin is full or not.
5 V power supply	-	Powers the microcontroller.
Camera		Captures input images.

Table 4 Software details

Name	Description
Wokwi	Online software for IoT circuit simulation.
ThingSpeak	Data aggregation and analytics. Integration of circuit to script.
TensorFlow, Keras	Used to create and train image identification model.
Pillow	Image processing library in Python
NumPy	Used for array operations.
Matplotlib	Plotting and visualization library. Used for accuracy.
TorchVision	Used for dataset access.
CV2	Used to integrate webcam for image input.

Table 5 Per-class performance metrics on the validation set

Waste Category	Per-class Accuracy	Precision	Recall	F1-Score
Plastic	97.79%	93.36%	93.73%	93.54%
Paper	98.12%	94.57%	95.26%	94.91%
Metal	98.19%	95.14%	94.00%	94.57%
Glass	98.05%	94.03%	95.09%	94.56%
Organic	98.79%	95.48%	96.35%	95.91%
E-Waste	98.32%	95.52%	93.42%	94.46%

training time than ResNet-50 and 24% faster than VGG-16, reflecting its computational efficiency. To adapt DenseNet-121 for waste classification, we froze all convolutional layers up to the final dense block, preserving pre-trained ImageNet features and reducing training time and overfitting risks.

Our framework's 94.6% accuracy projects a potential reduction of landfill-bound recyclables. Similarly, recycling efficiency could improve as compared to manual sorting since automation is cheaper than manual labour over extending periods of time. Moreover, our framework offers a complete AI-based waste classification solution that improves on most of the existing frameworks in terms of accuracy, flexibility, and automation. While earlier research often relied on sensor-based sorting or pre-trained deep learning models that only covered a few waste categories, our CPS and IoT based AutoSortBin is trained on images to classify six types of waste: Paper, Glass, Metal, Plastic, Organic, and E-Waste.

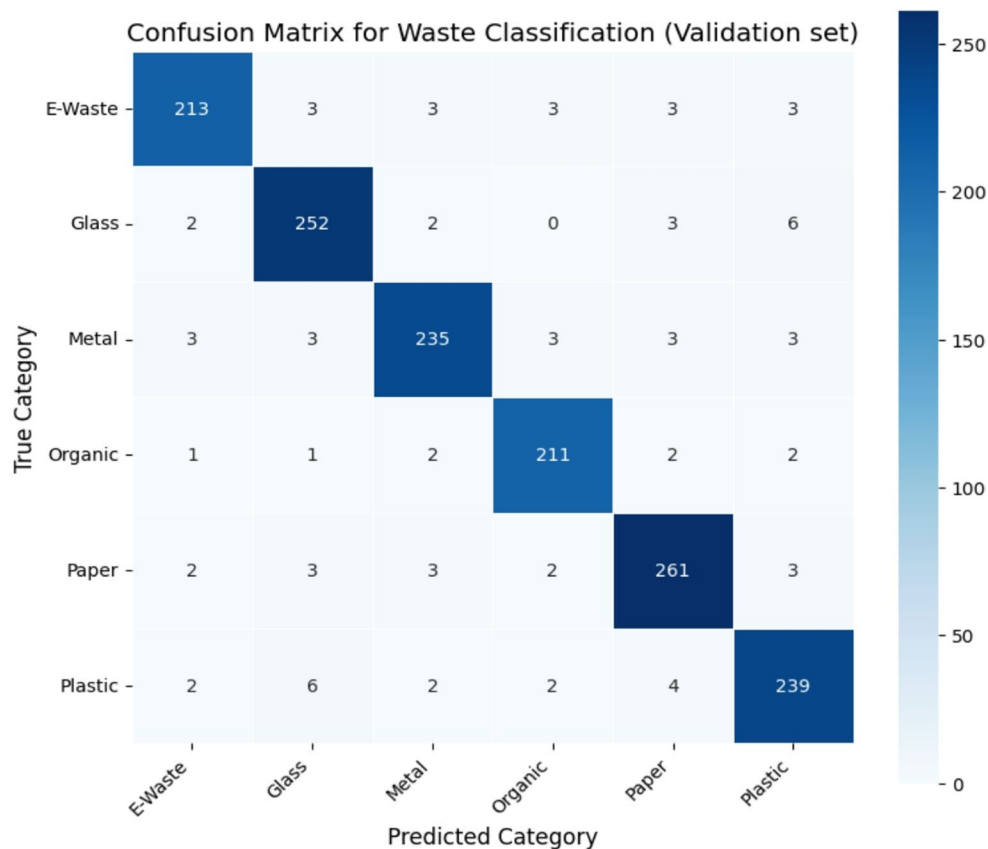
Notably, many earlier studies like those by Fernandez et al. [6], Das et al. [12], and Pereira et al. [14] did not handle e-waste at all, which is a growing concern today. AutoSortBin framework directly addresses this issue by considering e-waste making it sustainable as compared to existing frameworks. Although Song et al. [9] used EfficientNet-B0 for multi-class classification, it struggled with similar-looking materials. We tackled this by building a dataset focused specifically on waste types, which improved precision.

Moreover, our system supports real-time bin monitoring and sends instant email alerts when bins are full; a feature lacking in earlier works. This aids in distant management of the system which is a prominent feature in the CPS. This reduces delays, prevents overflow, and enables fully automated waste management. Unlike systems that just classify waste, AutoSortBin manages the disposal process from end to end; making it more efficient and

Table 6 Overall experimental results using DenseNet-121

Results	Training	Testing	Validation
Accuracy	94.68%	94.72%	94.63%
Precision	95.80%	95.50%	95.20%
Recall	93.90%	94.00%	93.70%
F1 Score	94.85%	94.74%	94.44%

Fig. 7 Confusionmatrix of the DenseNet-121 model on the validation set



practical for real-world use. A quantitative comparison of accuracy of AutoSortBin against previous contributions is depicted in the Fig. 8.

We selected these related works because they reflect a variety of recent approaches to waste classification from Bonala et al. [7] with MobileNet for recyclable detection to Song et al. [9] using EfficientNet for multi-class waste types. These papers offered a balanced mix of traditional and AI-driven techniques, helping us benchmark against both basic and advanced systems. We compared works using multiple

benchmarks: classification accuracy (%), waste category coverage, and real-time monitoring.

Conclusion and Future Work

Improper waste disposal leads to pollution and remains a major global concern. Most waste is still mismanaged or not segregated properly, with current methods being outdated and inaccurate. During development, we found that using a

Table 7 Sample of real-time waste detection

Image	Output	Image	Output	Image	Output	Image	Output
	E-Waste		Paper		Plastic		Glass
	Paper		Organic		Paper		Metal
	E-Waste		Plastic		Metal		Organic

Table 8 Comparison with baseline transfer learning models

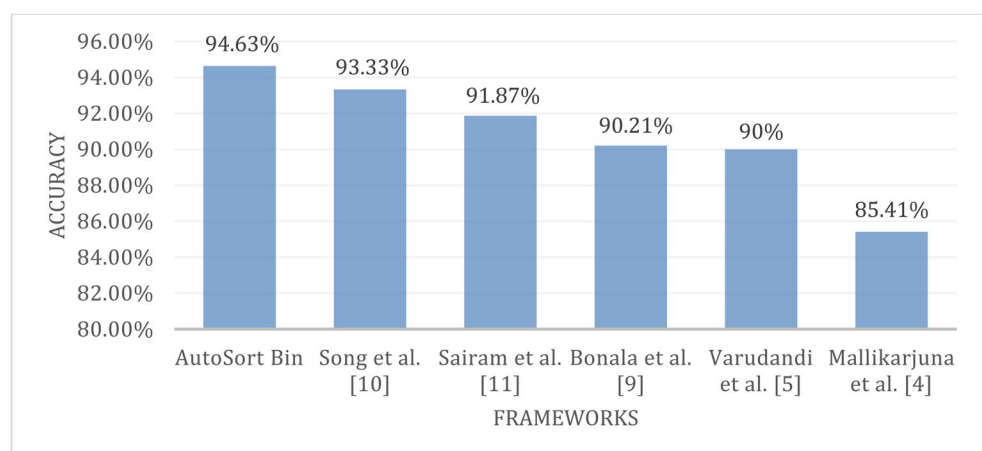
Model	Accuracy	F1 Score	Training Time (sec)
VGG-16	91.1%	89.3%	≈ 6,797
ResNet-50	92.8%	91.8%	≈ 5,870
DenseNet-121	94.6%	94.4%	≈ 5,165

custom dataset improved classification accuracy, especially for similar-looking waste types. Adding e-waste handling, real-time monitoring, and CPS-based automation made the system more practical and sustainable. AutoSortBin is a CPS and IoT-enabled waste segregation system that classifies waste into six categories namely e-waste, glass, metal, organic, paper, and plastic using transfer learning with DenseNet-121. The classified data is sent via ThingSpeak to an ESP32 circuit for fast, automated disposal, while also providing system notifications and email alerts for full bins. By automating classification and actuation, the CPS-IoT system reduces human involvement in segregation, improving throughput and minimizing misclassification errors. Compared to earlier works, AutoSortBin offers broader category coverage, real-time operation, and better automation, making it suitable for residential areas, industries, and public spaces. AutoSortBin directly supports smart city initiatives by enabling automated, data-driven waste segregation and monitoring. It also aligns with the UN Sustainable Development Goals (SDGs), particularly SDG 11 (Sustainable Cities and Communities), SDG 12 (Responsible Consumption and Production), and SDG 13 (Climate Action). Challenges like sensor power usage and e-waste from electronics remain, but future upgrades like GSM modules and better sensors can make it even more efficient.

The framework is highly scalable and modular, allowing easy addition of bins, sensors, or processors. Scaling AutoSortBin for use in societies, cities, or industries requires upgrades in hardware, software, and operations. Expanding beyond six waste types needs more bins, sensors, and durable components, along with a stronger microcontroller like Raspberry Pi. AutoSortBin's viability depends on energy

use, cost, and maintenance. The ESP32 enables low power consumption, making the system energy-efficient, and large-scale deployments can reduce costs through bulk procurement and modular hardware upgrades. Routine maintenance mainly involves sensor recalibration and occasional motor replacement, which can be scheduled predictively to minimize downtime. Software scalability involves retraining the DenseNet-121 model for new waste types and shifting from ThingSpeak to robust platforms like AWS or Azure.

AutoSortBin lays a strong foundation for smarter waste management but still has room to grow. Using edge devices like Raspberry Pi or Jetson Nano can make the system faster by handling processing locally. Lighting variations and partial occlusion can reduce recognition accuracy, while continuous operation may affect actuator durability. To address these, future works will include testing varied conditions, low-cost illumination to stabilize inputs, and integration of auxiliary sensors (e.g., weight or RFID) for improved reliability. While AutoSortBin shows 94.63% accuracy on curated data and simulated tests, we recognize that uncontrolled conditions may differ. A practical pilot study with mixed waste and varied environments is planned as future work, and its results will be reported separately. Better sensors like lidar or infrared, along with LED lighting, can help it work more reliably in different environments. Work can be done to include mixed-waste experiments to assess classification and long-term testing to evaluate model performance. We can also expand the number of waste categories by using larger datasets and advanced techniques like multi-label classification. To enhance scalability, predictive analytics can use historical bin data to forecast fill patterns and optimize collection routes, while blockchain can securely log bin events for

Fig. 8 Accuracy comparison of the proposed AutoSortBin framework with the existing frameworks

tamper-proof, auditable tracking. Additional improvements include solar-powered operation with integrated panels and batteries for energy independence, and mobile app integration to provide real-time bin status, notifications, and analytics for users and municipal authorities. For places with poor Wi-Fi, adding GSM or newer IoT networks like LoRaWAN can help send data over long distances with low power.

Author Contributions Mayank Sangwan and Avin Chaudhary conceptualized the AutoSortBin framework and developed the IoT integration on the Wokwi simulator and managed the hardware simulation setup. Ayush Gupta and Himank Gupta designed and implemented the machine learning model using DenseNet-121 and led the manuscript writing. Vinay Kharb reviewed related papers, collected and annotated the custom dataset and contributed to the experimental evaluation. Shweta Sharma and Sweeti Sah supervised the overall research and provided critical revisions to the manuscript. All authors reviewed and approved the final manuscript.

Funding There is no funding from any organization. We are submitting our manuscript from 5th International Conference on Computing Analytics and Networks 2025 (ICAN 2025 Conference), Chitkara University, Himachal Pradesh.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. The World Bank (2022) Solid waste management. <https://www.worldbank.org/en/topic/urbandevelopment/brief/solid-waste-management>. Available: Onlines Accessed 18 Apr 2025
2. United Nations Environment Programme (UNEP) (2022) Beat plastic pollution. <https://www.unep.org/interactives/beat-plastic-pollution>. Available Online Accessed 18 Apr 2025
3. Huang G, Liu Z, Van Der Maaten L, Weinberger KQ (2017) Densely connected convolutional networks, in Proceedings of the international conference on computer vision and pattern recognition (CVPR), Honolulu, HI, USA, pp 2261–2269. <https://doi.org/10.1109/CVPR.2017.243>
4. Wokwi (2025) ESP simulator - Wokwi. <https://wokwi.com>. Available:Online Accessed 18 Apr 2025

5. MathWorks (2025) ThingSpeak – IoT Platform. <https://thingspeak.mathworks.com>. Available Online Accessed 18 Apr 2025
6. Fernandez I G, Neog A, Singh A, Singh A, Bharath A (2024) Smart-segregation of waste management using IoT. Proceedings of the 9th international conference on science technology engineering and mathematics (ICONSTEM), Chennai, India, pp 1–8. <https://doi.org/10.1109/ICONSTEM60960.2024.10568703>
7. Bonala K, Saggurthi P, Kambala PK, Voruganti S, Utukuru S, Sugamya K (2024) Efficient Handling of Waste Using Deep Learning and IoT. Proceedings of the 2nd international conference on sustainable computing and smart systems (ICSCSS), Coimbatore, India, pp. 368–373. <https://doi.org/10.1109/ICSCSS60660.2024.10625621>
8. Kabilan B, Sairam R, Praveen M (2024) Enhanced CNN Architecture for accurate waste classification in smart cities, in proceedings of the international conference on IoT based control networks and intelligent systems (ICICNIS), Bengaluru, India, pp. 538–543. <https://doi.org/10.1109/ICICNIS64247.2024.10823383>
9. Song Y, He X, Tang X et al (2024) DEEPBIN: deep learning based garbage classification for households using sustainable natural technologies. J Grid Comput 22:2. <https://doi.org/10.1007/s10723-023-09722-6>
10. Varudandi S, Mehta R, Mahetalia J, Parmar H, Samdani K (2021) A smart waste management and segregation system that uses Internet of Things, machine learning and Android application, in Proceedings of the 6th International Conference for Convergence in Technology (I2CT), Maharashtra, India, pp. 1–6. <https://doi.org/10.1109/I2CT51068.2021.9418125>
11. Fathimal SNPM, R. R. and, Prakash K (2019) Smart garbage segregation & management system using Internet of Things (IoT) & Machine Learning (ML), in Proceedings of the 5th International Conference on Advanced Computer & Communication Systems (ICACCS), Coimbatore, India, pp. 287–290. <https://doi.org/10.1109/ICIICT1.2019.8741443>
12. Das A, Shukla A, Manjunatha R, Lodhi EA (2021) IoT-based solid waste segregation using relative humidity values. Proceedings of the 3rd international conference on intelligent communication technologies and virtual mobile networks (ICICV), Tirunelveli, India, pp. 312–319. <https://doi.org/10.1109/ICICV50876.2021.9388611>
13. P MGC, Yadav S, Shanmugam A, H. V., and, Suresh N (2021) Waste classification and segregation: Machine learning and IoT approach. Proceedings of the 2nd International Conference on Intelligent Engineering and Management (ICIEM), London, United Kingdom, pp. 233–238. <https://doi.org/10.1109/ICIEM51511.2021.9445289>
14. Pereira W, Parulekar S, Phaltankar S, Kamble V (2019) Smart bin (waste segregation and optimisation). Proceedings of the amity international conference on artificial intelligence (AICAI), Dubai, United Arab Emirates, pp. 274–279. <https://doi.org/10.1109/AICAI.2019.8701350>
15. Azkefe F (2022) TrashNet. <https://doi.org/https://www.kaggle.com/datasets/feyzazkefe/trashnet>. Available Online Accessed 18 Apr 2025
16. Abila M (2023) Garbage classification. <https://www.kaggle.com/datasets/mostafaabla/garbage-classification>. Available Online Accessed 18 Apr 2025
17. Sharma S, Krishna CR, Kumar R (2021) RansomDroid: forensic analysis and detection of android ransomware using unsupervised machine learning technique. Forensic Science International: Digital Investigation. <https://doi.org/10.1016/j.fsidi.2021.301168>
18. Wang X, Zhao F, Lin P, Chen Y (2022) Evaluating computing performance of deep neural network models with different backbones on IoT-based edge and cloud platforms. Internet Things 20:Art100600. <https://doi.org/10.1016/j.iot.2022.100609>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.